

Publish AI, ML & data-science insights to a global community of data professionals.

[LATEST](#) [EDITOR'S PICKS](#) [DEEP DIVES](#) [NEWSLETTER](#) [WRITE FOR TDS](#)



Claude in Microsoft Foundry

New Model Mondays on demand—Hands-On with Claude in Microsoft Foundry.

[Watch now](#)

BUSINESS

# Machine learning that pays the bills: choosing models in business contexts

Sophisticated models look good on research papers but might not pay off in business use cases

Fabio Baccarin

Dec 28, 2020 17 min read



Photo by [Scott Graham](#) on [Unsplash](#)

## Introduction

My main point in this article is that in business contexts we should not choose between machine learning models based on predictive performance alone. I approach the problem of choosing models from the investment perspective: different models are alternative courses of action, each with some kind of

cost and benefit attached. As with other investments facing businesses, we need to try to measure if the benefits outweigh the costs for each alternative to choose wisely between them.



Claude in Microsoft Foundry

New Model Mondays on demand—Hands-On with Claude in Microsoft Foundry.

[Watch now](#)

measured their predictive performance against the time we need to fit them and make predictions. I found out that a model like LightGBM could not compete with models like logistic regression or Gaussian Naïve Bayes (GNB). The improvement in predictive performance for LightGBM is not enough to compensate for the extra costs of fitting it and making predictions.

I used a simple analysis called cost-effectiveness analysis, which is similar to the more popular cost-benefit analysis. So, before discussing the results, I briefly explain what these analyses are and how we can use them. So, without further ado, let's get to it.

## Cost-benefit analysis: trading chess pieces

I think the best example of how cost-benefit analysis is present in our lives and makes much sense is in chess. In this game, pieces have different values called *points*. For example, the Queen is the most powerful piece on the board and therefore worths 9 points. The Pawn is the weakest piece, so it worths just 1 point. It is very common in chess for two or more pieces to be under attack by the opponent's protected pieces, in such a way that a particular move will earn you a piece at the cost of losing another. Chess is a war game, and in war the army with more soldiers always have the tactical advantage. Therefore, the relative value of pieces is fundamental to the game, making it possible to tell who has the tactical advantage at any moment.

Because every piece is in a common unit of measurement, we can just subtract the cost (the value of the piece I lose) from the benefit (the value of the piece I earn) for a given move. For example, one should rarely move the Queen in such a way that it gets captured by a Pawn because this move results in a loss of 8 points. In other words, the cost of losing the Queen doesn't compensate for the benefit of earning a Pawn, so that the player making such a move ends up in a tactical disadvantage equivalent to 8 points. By making such calculations for many different moves, the chess players are able to choose wisely between different courses of action, gaining a tactical advantage or compensating for the opponent's.

This is all there is to know about cost-benefit analysis. It is very intuitive and easy to do. But it has many problems. One of them is that cost-benefit analysis can be used only if we can measure things in the same unit, that is if things are *commensurable*. To solve this problem, we need to find a way to compare incommensurable things. For this, we have cost-effectiveness analyses.

## **Cost-effectiveness analysis: trading money for electricity**

One of the most common applications of cost-effectiveness analysis is in the energy industry. Getting the cost of providing energy from a given source is relatively simple because we can measure the costs along the chain and arrive at some estimate. Guessing future energy prices, however, is a very tricky job because they change rather chaotically all the time. So we have to measure energy in another way, such as kilowatt-hour (denoted kWh), which is much easier to do.

Now that we have costs in money and energy in kWh, how we calculate the equivalent of the cost-benefit measure? Because the two quantities are in different units, we cannot subtract them: subtracting money from kWh doesn't make any sense. What we can do instead is to divide one by the other, getting the *cost-effectiveness ratio*. This ratio can be used to compare different sources of energy to determine which one is better.

For example, let's say we want to decide whether to use coal or wind to produce energy. So, we can reason about that problem in the following terms: if I divide the amount of money spent to get the observed amounts of energy for both sources, the result is how much money I need to spend to get 1 kWh of energy from that source. The source with the lowest money/kWh ratio is the best because it is the one in which we spend less money to get the same amount of energy.

So, consider the completely hypothetical values in the table below. There we can have an idea about why most countries in the world still use pollutant energy sources. Coal requires \$6.50 to produce 1 kWh, while wind requires twice that. So eolic energy is twice more expensive in this scenario, even after accounting for the environmental damage of coal energy. Therefore it is more effective to use coal and then do damage control than to use eolic energy. For example, we could use the money earned from

the 30% fee on coal energy producers to fund research in eolic energy. This way, eventually engineers will be able to bring the costs down enough to make eolic energy as efficient as coal energy (or bring the energy produced up).

| Source | Production costs (\$) | Environmental fee (%) | Total cost (\$) | Energy (kWh) | Cost-effectiveness raio (\$/kWh) |
|--------|-----------------------|-----------------------|-----------------|--------------|----------------------------------|
| Coal   | 5.000,00              | 30%                   | 6.500,00        | 1.000,00     | 6,50                             |
| Wind   | 7.000,00              | 0%                    | 7.000,00        | 538,00       | 13,01                            |

A completely hypothetical example of cost-effectiveness analysis. Made by the author

## Applications to machine learning

Now that you got the idea of cost-effectiveness analysis, let's turn to the problem at hand. I have a classification problem on a dataset with 595,212 rows and 57 features. But here what I'm trying to figure out is not the model that will win the Kaggle competition, but what model Porto Seguro should adopt to earn the best financial return.

The business scenario I'm imagining is the one in which models will be trained and deployed in cloud environments, where the main cost is the fee on computing power. Because those fees are charged by the hour, time is the most convenient unit to measure costs. On the other hand, raw predictive power is the most convenient unit for measuring the benefits (or effects). Here, my chosen metric is the average precision, also known as *positive predictive value* (PPV). I prefer this metric to the more popular AUC-ROC because it has a nice business interpretation. If I have a 1% PPV, it means that 1 out of 100 clients expected to file an insurance claim will actually do so. To make the discussion clearer, I will drop the percentage and just say that 1 PPV equals 1 true-positive out of 100 positives. This way, 0.1 PPV equals 1 true-positive out of 1,000, 0.01 PPV equals 1 true-positive out of 10,000, and so on. False-negatives are already accounted for because of the way Scikit-Learn calculates the average precision (check it out [here](#)).

For brevity, I will skip the details about what I did to arrive at my model results, but you can check out the full analysis and code in the [project repository on GitHub](#). All that you need to know is that this analysis requires that all models be ran in the same hardware (at least with the same amount of CPUs and RAM memory). Otherwise, the time measurements (our proxy for costs) won't be fair because some models would have more computing power at their disposal than others. One way to account for this is to multiply by the fee charged by the cloud provider to use the machine, but this is an unnecessary

complication. Using the same machine is much easier. Also, I produced the results discussed below after grid-searching. So the measurements aren't influenced by inefficient hyperparameter configuration. Finally, I also incorporated all preprocessing into every model and all of them use the same preprocessing pipeline. This way the cost of preprocessing is accounted for and there is no difference in preprocessing between models.

I computed averages and standard errors from 50 cross-validations of every model (specifically, repeated stratified cross-validation). I measured the time each model took to fit and predict, along with their respective average precision or PPV. The standard errors on those averages are meant to give an idea of the degree of uncertainty in those measurements. These results are in the table below.

| Model                 | Mean fit time (ms) | Mean score time (ms) | Mean test average precision (%) | Fit time standard error (ms) | Score time standard error (ms) | Test average precision standard error (%) | Cost-effectiveness ratio |
|-----------------------|--------------------|----------------------|---------------------------------|------------------------------|--------------------------------|-------------------------------------------|--------------------------|
| naive_bayes_gaussian  | 4243.83            | 590.82               | 5.33                            | 26.18                        | 6.30                           | 0.02                                      | 906.49                   |
| naive_bayes_bernoulli | 4352.76            | 598.38               | 5.26                            | 36.87                        | 11.34                          | 0.02                                      | 940.40                   |
| logistic_regression   | 8819.66            | 463.18               | 5.40                            | 56.08                        | 5.74                           | 0.02                                      | 1720.43                  |
| lasso                 | 9396.20            | 493.42               | 5.40                            | 104.76                       | 8.26                           | 0.02                                      | 1832.85                  |
| elastic_net           | 9428.55            | 493.78               | 5.40                            | 129.02                       | 9.70                           | 0.02                                      | 1838.91                  |
| ridge                 | 9528.90            | 505.77               | 5.40                            | 125.03                       | 10.29                          | 0.02                                      | 1859.75                  |
| decision_tree         | 10178.74           | 513.50               | 3.68                            | 134.34                       | 6.32                           | 0.00                                      | 2905.86                  |
| light_gbm             | 35080.39           | 1787.03              | 5.43                            | 138.69                       | 17.90                          | 0.02                                      | 6795.69                  |

Cross-validation results for different models on Porto Seguro's Safe Driver dataset. Time measurements might change depending on the hardware the experiments were conducted. Made by the author.

Here we confirm what I said in the introduction: LightGBM won't return more money to the company than GNB. GNB costs 906.5 milliseconds (ms) per PPV, meaning that it costs that much time to find 1 true-positive out of 100 positives (notice how the choice of an interpretable metric facilitates our analysis). On the other hand, LightGBM costs almost 6,800 ms/PPV, which is about seven and a half times higher. This corresponds to an increase in costs of about 650%! Training and predicting with LightGBM is seven and a half times more expensive than training and predicting with GNB. This means that LightGBM uses seven and a half more resources to find the same amount of true-positives as GNB, even after accounting for the fact that its PPV is on average higher than GNB's.

I can't dispute the fact that LightGBM is better at predicting than GNB. After all, LightGBM has 5.43 PPV against 5.33 PPV for GNB. Given a standard error of 0.02 PPV, this 0.1 PPV difference is statistically significant, meaning it would probably be observed if we repeated this comparison many times with similar data. Many

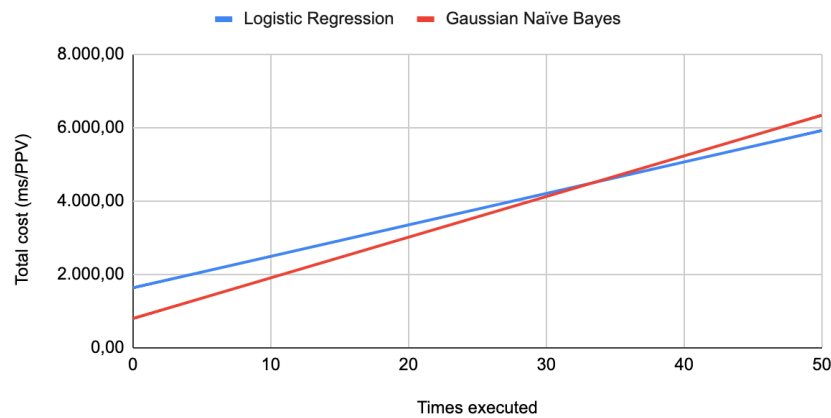
data scientists don't even bother to look at standard errors for doing such analysis. But for those that do, it is important to also consider the magnitude of the difference observed. This is because statistical significance is designed only to tell if results are reproducible, not if they are practically relevant. The standard error always shrinks as the sample size grows, thus making every difference, however small, statistically significant. If you have a large enough sample, ridiculously small differences would be highly statistically relevant. This is the reason why a mere 0.1 PPV increase corresponds to five standard errors, enough to run a test with less than 1% probability of a false-positive. But this translates into only one more true-positive out of 1,000! Does this really make a difference to the business? Probably not if every such positive is, in this case, seven and a half times more expensive to find.

So, what is needed for LightGBM to be cost-effective? Two things can happen to accomplish this here. First, folks that developed LightGBM could find out a way of dropping both training and prediction time, so that LightGBM trains and predicts as fast as GNB (and therefore as cheaply). Alternatively, the PPV of LightGBM should increase by over seven and a half times, reaching over 40 PPV, so that it is so much better at predicting that it compensates for the additional costs. But LightGBM is only about 1.88% better than GNB at predicting and this is the reason it is the least cost-effective.

If we look closely at this table, we can spot another interesting thing one should think about when doing this analysis. Look at logistic regression: it is nearly twice more expensive to train, but about 22% cheaper to predict. This means that, eventually, the logistic regression will catch up with GNB by returning the extra training time in the form of prediction time saved during job execution.

Let's think about that. Logistic regression costs about 1,633 ms/PPV to train and about 86 ms/PPV to predict every time we run the job. On the other hand, GNB costs about 796 ms/PPV to train and about 111 ms/PPV to predict. So, how should the cost evolve in a scenario where we trained each model one time, and then predicted with it 50 times? It should evolve like this:

## Evolution of cost for Logistic Regression and Gaussian Naïve Bayes



Total cost incurred by times executing the prediction job. Logistic regression (blue line) eventually compensates for its higher training time in the form of a faster prediction. The Gaussian Naïve Bayes (red line) starts with advantage but progressively loses it due to its slower prediction. Made by the author.

The logistic regression (blue line) starts higher because of its higher training cost. But the cost grows slower than that of GNB (red line); therefore, sooner or later the cost curve of GNB will cross the cost curve of logistic regression, making it more expensive. I assumed here that there is no time value of money, so that the discount rate is zero and the curves evolve linearly. In this scenario, the logistic regression would catch up in the 34th time we ran the prediction job. From then on, it will save us money in computational fees. The higher the discount rate, the longer it takes logistic regression to achieve this. Hence assuming a zero discount rate both simplifies calculations and makes the analysis more conservative by giving an advantage to the challenger model.

This leads us to think about the expected lifetime of the model. That is, about the time we expect would have to pass before we need to retrain the model. If the prediction job runs monthly, it would take nearly three years for logistic regression to catch up. By then, we would probably have retrained the model already. So logistic regression would never invert the balance. But if the job runs daily, in a little over a month logistic regression will pay off, for we probably won't retrain in the next month. Note that LightGBM will never pay off because it is more expensive both to train and to predict than those two alternatives.

## Conclusion

There are two main takeaways from this study. The first is that, in business contexts, choosing between machine learning models transcends the evaluation of predictive performance. The

measurement of improvement in predictive performance may be very reliable (i.e., statistically significant), but it might not be enough to compensate for increased costs or even to make a significant impact on the business. By studying Porto Seguro's Safe Driver dataset, I found out that using LightGBM would probably result in a loss of money, for this model is about seven and a half times more expensive than GNB. The difference in PPV between LightGBM and GNB is surely statistically significant but corresponds to only one more true-positive out of 1,000. This raises the question of whether LightGBM impacts the business in a manner worthy of its costs.

The second point is that cost-benefit and cost-effectiveness analyses are simple but powerful tools. However, this does not mean we don't need to be cautious in using them. Cost-effectiveness analysis solves the problem of incommensurability, that is, it allows us to compare different things. It is therefore more general than cost-benefit analysis, which requires costs and benefits to be in the same unit of measurement. Yet both of them suffer the problem of inaccurate calculations and, more importantly, from the omission of relevant factors. Some of these factors may be impossible to measure in any way at all. Hence this kind of analysis is only fruitful if done with much critical reasoning.

Cost-effectiveness and cost-benefit analyses are guides to good decisions. But good decisions don't come from blindly following numbers. They come from careful thinking. Good chess players don't blindly follow piece trade calculations, but make carefully thought decisions using them. So do good political leaders about money/kWh ratios. It is perfectly fine for a chess player to trade the Queen for a Pawn if he or she is sure it will lead to a checkmate. Winning the game brings much more benefit than it costs to lose 8 points, even if this doesn't enter into the calculation because the game itself can't be measured in piece points. Similarly, a good political leader may choose to stick with eolic energy to make a strong political point about the importance of adopting less pollutant energy sources. Every decision boils down to what you are trying to accomplish. If you get what you wanted, you made a good decision. It is as simple as that.

So, are we making good decisions about which model to adopt in business contexts? The very simple case study I did here hints that we tend not to do so. At least, I have never seen anyone talk about how much it costs to have a sophisticated model in

production in the way I did here. Actually, I did this case study purely out of curiosity, for I asked myself that many times recently and wanted to have some hint at the answer.

Businesses thrive by making fruitful investments. Machine learning is a particularly expensive investment: it requires many highly capable professionals, a highly sophisticated infrastructure, and a mindset very few companies actually have. So data scientists and machine learning engineers should be very careful in deciding which model to adopt. Looking only to raw predictive power may not be enough in such situations.

I'm not saying here that we should never consider sophisticated models. I personally dislike Naïve Bayes, but I'm a data-driven person and for this particular problem I would go with GNB. Because I carefully thought about its pros and cons and systematically compared them to alternative models, aiming at what really matters for the job I wanted to do.

If LightGBM exhibited the best trade between costs and predictive power, I sure would go with it. The best model is always the one that serves its purpose best. The point is that this purpose might not be wholly incorporated in a predictive metric, thus creating the need for a more business-centric analysis. Many machine learning and data science learning material are academically inclined and therefore don't address points like these. That is perfectly fine: their authors simply didn't want to address them, choosing to focus on other matters they thought more important. That is a very legitimate reason and I myself would choose to do so if I were in their place.

Also, it is perfectly fine if facing this data you still want to go on with LightGBM (or any other model in this specific situation). But would you also pay seven and a half times more for a data scientist that is only 1.88% more productive? If so, well, you are consistent and that is all that matters. If not, you should stop and think about that. It is fine if you want to have sophisticated models in production and hence choose to ignore their inefficiency (if it exists). In the same way, it is fine if the mentioned data scientist is your nephew and you don't care about nepotism (and neither if others say you have dubious moral values). Accept or reject the conclusion, either way you are still being more careful than blindly following higher predictive performance measures. The purpose of such analyses is exactly that: to induce careful thinking. As long as a decision is a carefully thought one, it is as good as any decision can be.

WRITTEN BY

Fabio Baccarin

See all from Fabio Baccarin

Business

Data Science

Machine Learning

Share This Article



**Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.**

Write for TDS

## Related Articles



ARTIFICIAL INTELLIGENCE

### Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



ARTIFICIAL INTELLIGENCE

### How to Forecast Hierarchical Time Series

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 13 min read



DATA SCIENCE



MACHINE LEARNING